

Experiment 16 — Comparison of Classification Algorithms

AIM

To implement and compare different classification algorithms and evaluate their performance using a suitable dataset.

SOFTWARE USED

- Python
- Google Colab / Jupyter Notebook
- NumPy
- Pandas
- Scikit-learn
- Matplotlib
- Seaborn

PROCEDURE

1. Open Google Colab and create a new notebook.
2. Import the required Python libraries.
3. Load the built-in Iris dataset.
4. Separate the input features and target classes.
5. Split the dataset into training and testing sets.
6. Implement different classification algorithms:
 - Logistic Regression
 - K-Nearest Neighbors (KNN)
 - Decision Tree
 - Support Vector Machine (SVM)
 - Naive Bayes
7. Train each classifier using the training dataset.

8. Predict the classes using the testing dataset.
9. Calculate the accuracy of each algorithm.
10. Store and compare the accuracy values.
11. Display the results using a bar chart.

CODE

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
# Define classification algorithms
models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "KNN": KNeighborsClassifier(n_neighbors=5),
```

```
"Decision Tree": DecisionTreeClassifier(random_state=42),
"SVM": SVC(),
"Naive Bayes": GaussianNB()
}
# Store accuracy values
results = {}
# Train and evaluate each model
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy
    print(name, "Accuracy:", accuracy)
# Display comparison
print("\nAlgorithm Performance:")
for name, accuracy in results.items():
    print(f'{name}: {accuracy:.2f}')
# Visualization
plt.figure(figsize=(10, 5))
plt.bar(results.keys(), results.values())
plt.xlabel("Classification Algorithm")
plt.ylabel("Accuracy")
plt.title("Comparison of Classification Algorithms")
plt.xticks(rotation=20)
plt.ylim(0, 1.1)
plt.show()
```

OUTPUT

Logistic Regression Accuracy: 1.0

KNN Accuracy: 1.0

Decision Tree Accuracy: 1.0

SVM Accuracy: 1.0

Naive Bayes Accuracy: 1.0

Algorithm Performance:

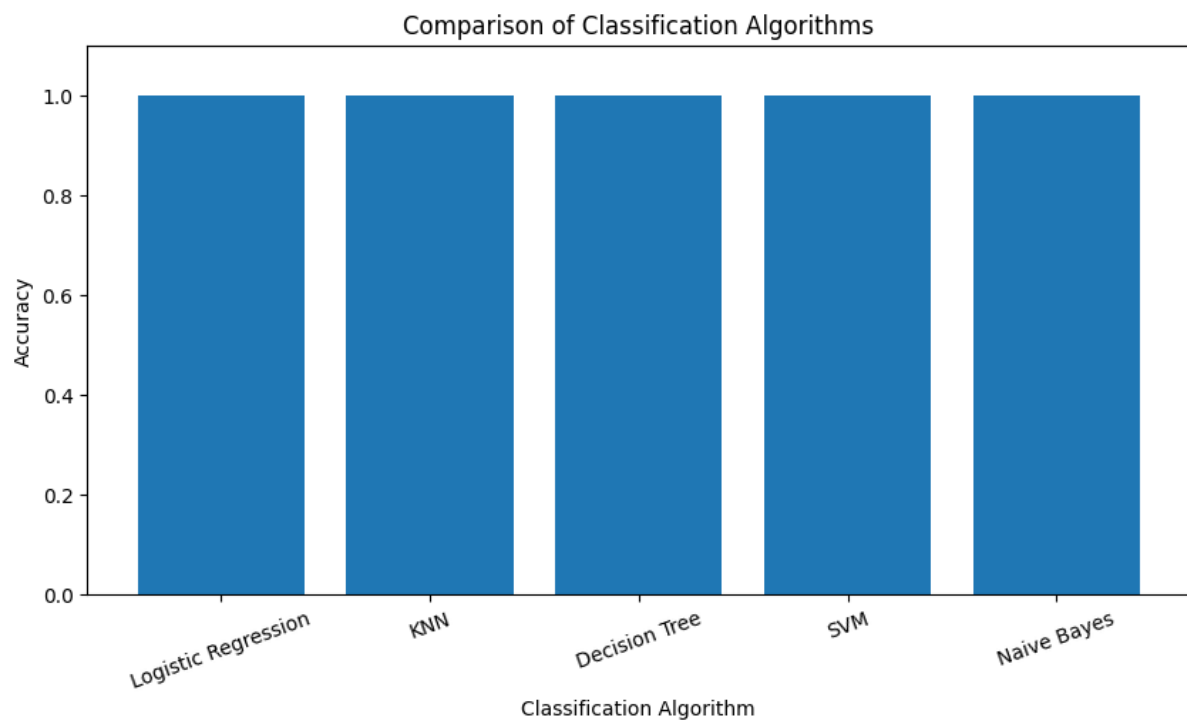
Logistic Regression: 1.00

KNN: 1.00

Decision Tree: 1.00

SVM: 1.00

Naive Bayes: 1.00



RESULT

Thus, different classification algorithms were successfully implemented and their performances were compared using accuracy, and the results were visualized using a bar chart.